



Full-Stack Technical Decisions Document

React · NestJS · MongoDB · NX Monorepo

1. Monorepo — NX

- NX workspace root • apps/frontend (React + Vite) • apps/backend (NestJS) • libs/validation (shared Zod schemas) • libs/types (shared TS interfaces)
- Incremental build cache • Affected-only test runs • Enforced module boundaries • Single tsconfig / ESLint root • README.md + AI.md at root

2. Frontend Stack

- React 18 + TypeScript • Vite (via NX) • Redux Toolkit + RTK Query • React Router v6 • Zod (shared from libs/validation)
- Sign Up page — full field validation • Sign In page • Protected App/Welcome page • Auth guard HOC / route wrapper • Global error boundary

3. Backend Stack

- NestJS + TypeScript • Mongoose ODM → MongoDB • nestjs-zod (Zod ↔ NestJS pipes) • Passport.js + JWT strategy • @nestjs/throttler (rate limiting) • class-validator (secondary guard)
- POST /auth/register • POST /auth/login • GET /auth/me ← protected • Global validation pipe • Global exception filter • Swagger via @nestjs/swagger

4. Shared Validation — Zod

All validation schemas live in **libs/validation** and are imported by both apps. This guarantees FE and BE enforce identical rules with zero duplication.

XSS sanitisation is applied inside `Zod .transform()` — strips `<script>`, `<img onerror>`, and other dangerous tags before data reaches any handler. `nestjs-zod` bridges the schemas to NestJS pipes automatically.

5. Security

Area	Decision	Rationale
Passwords	bcrypt (salt rounds 12)	Industry standard; irreversible hashing
Auth tokens	JWT (HttpOnly cookie)	Prevents JS access; short expiry + refresh pattern
CSRF	Double-submit cookie	csrf middleware; token validated on state-changing routes
XSS	Zod .transform() sanitise	Strips dangerous HTML tags at schema level, both FE & BE
Rate limit	@nestjs/throttler	Per-IP limits on /auth/* endpoints; configurable TTL & limit
Headers	Helmet.js	Sets secure HTTP headers (CSP, HSTS, X-Frame-Options ...)

6. Logging & Observability

Library: nestjs-pino (wraps pino, fastest Node.js logger)

Area	Decision	Rationale
Structured logs	nestjs-pino	JSON output; every line tagged with level, route, ms
Request tracing	Correlation ID middleware	UUID injected via AsyncLocalStorage; propagated to all log lines for a single request
Dev pretty-print	pino-pretty	Pipe: nx serve backend pino-pretty — coloured, readable
Dev UI (optional)	Seq (Docker)	docker run datalust/seq → localhost:80; filter by requestId; zero infra

Sample correlated log journey — all four lines share the same requestId:

```
{ "requestId":"abc-123", "msg":"Request received", "route":"/auth/login" }
{ "requestId":"abc-123", "msg":"Validating payload" }
{ "requestId":"abc-123", "msg":"User not found", "level":"warn" }
{ "requestId":"abc-123", "msg":"Response sent", "status":401, "ms":12 }
```

7. Testing

Area	Decision	Rationale
Backend unit	Jest (built-in NestJS)	Services, guards, pipes; mocked Mongoose models
Backend e2e	Jest + Supertest	Full HTTP layer; test containers for MongoDB
Frontend unit	Vitest	Fast, Vite-native; component & hook tests
Frontend e2e	Cypress	Full sign-up / sign-in / protected route flows

8. API Documentation — Swagger

@nestjs/swagger + **nestjs-zod** — Zod schemas auto-generate OpenAPI models. Available at /api/docs in development.
Decorators: @ApiTags, @ApiOperation, @ApiBearerAuth on protected routes.

9. Docker & Dev Environment

docker-compose.yml (dev)

Area	Decision	Rationale
mongo	mongo:7	DB on port 27017; named volume for persistence
seq	datalust/seq	Log UI on port 80; optional, dev only
backend	Dockerfile.backend	NestJS; hot-reload via NX serve; depends_on mongo
frontend	Dockerfile.frontend	React/Vite; depends_on backend

Dockerfile strategy

- Multi-stage build (dev / build / prod) • node:20-alpine base image • Dev stage: NX + ts-node + hot reload • .dockerignore excludes node_modules & dist
- Prod stage: compiled JS only, no devDeps • Non-root USER for security • HEALTHCHECK on /health endpoint • ENV vars via .env.docker (gitignored)

10. Full Decision Summary

Area	Decision	Rationale
Monorepo	NX	Caching, boundaries, shared libs
Frontend	React 18 + TypeScript	Task requirement; Vite via NX

Backend	NestJS + TypeScript	Task requirement; decorator-based, testable
Database	MongoDB + Mongoose	Task requirement; flexible schema
Validation	Zod (shared lib)	Single source of truth FE + BE
NestJS bridge	nestjs-zod	Zod → pipes + Swagger schemas
State mgmt	Redux Toolkit + RTK Q	Auth state + API layer
Auth	JWT (HttpOnly cookie)	Secure token storage
CSRF	Double-submit cookie	Stateless CSRF protection
XSS	Zod .transform()	Schema-level sanitisation
Rate limiting	@nestjs/throttler	Guard /auth/* endpoints
HTTP headers	Helmet.js	Secure headers out of the box
Logging	nestjs-pino	Structured JSON + correlation IDs
Dev log UI	Seq (Docker)	Local requestId journey viewer
BE tests	Jest + Supertest	Unit + e2e built into NestJS
FE tests	Vitest + Cypress	Unit + full e2e flows
API docs	Swagger (@nestjs/swagger)	Auto from Zod schemas
Containers	Docker Compose	Mongo + Seq + apps in one command